Git Workflow Examples — Real Scenarios

Scenario A: Branch Off `main`

This is the most common setup. Your team says: "Branch off main."

```
main:  A ── B ── C ── D        ← production code
                      │
your branch:          └── E ── F    ← your work
```

Step by step

```
# 1. Switch to main
git checkout main

# 2. Get the latest code from everyone
git pull origin main

# 3. Create YOUR branch
git checkout -b feature/add-email-validation

# 4. You're now on your branch. Verify:
git branch
#   main
# * feature/add-email-validation    ← you are here

# 5. Do your work (edit files)...

# 6. Check what you changed
git status
```

What `git status` looks like

```
On branch feature/add-email-validation
Changes not staged for commit:
        modified:   app/routes/users.py
        modified:   app/services/user_service.py

Untracked files:
        tests/test_email_validation.py
```

This tells you:

- 2 files were **modified** (existed before, you changed them)

- 1 file is **untracked** (brand new file you created)

What `git diff` looks like

```
git diff
```

```
diff --git a/app/routes/users.py b/app/routes/users.py
index 3a2b1c0..8f4e2d1 100644
--- a/app/routes/users.py
+++ b/app/routes/users.py
@@ -15,6 +15,8 @@
 @router.post("/", response_model=UserResponse, status_code=201)
 async def create_user(user: UserCreate, db: AsyncSession =
Depends(get_db)):
+    if not is_valid_email(user.email):
+        raise HTTPException(status_code=422, detail="Invalid email
format")
     return await user_service.create_user(db, user)
```

Reading the diff:

- Lines starting with `+` are what you **added** (green)
- Lines starting with `-` are what you **removed** (red)
- Lines with no prefix are **context** (unchanged, shown for reference)

Stage, commit, push

```
# Stage specific files
git add app/routes/users.py
git add app/services/user_service.py
git add tests/test_email_validation.py

# Or stage everything at once (after checking git status first!)
git add .

# Commit
git commit -m "Add email validation to user registration"

# Push your branch to GitHub
git push -u origin feature/add-email-validation
```

Then open a Pull Request on GitHub

```
base: main  ←  compare: feature/add-email-validation
```

```
Title: Add email validation to user registration
Description: Validates email format before creating user. Returns 422 for
invalid emails.
```

After PR is approved and merged

```
git checkout main
git pull origin main                          # now includes your merged
code
git branch -d feature/add-email-validation      # delete your local branch
```

Scenario B: Branch Off `develop`

Some teams use a `develop` branch as a staging area before `main`.

```
main:     A ── B ── C                    ← production (deployed)
                    │
develop:            └── D ── E ── F      ← next release (testing)
                                 │
your branch:                     └── G ── H    ← your work
```

The only difference: you start from `develop` instead of `main`.

Step by step

```
# 1. Switch to develop (NOT main)
git checkout develop

# 2. Get the latest
git pull origin develop

# 3. Create your branch FROM develop
git checkout -b feature/add-email-validation

# 4. Verify
git branch
#   main
#   develop
# * feature/add-email-validation    ← you are here, branched from develop

# 5. Do your work, same as before...

# 6. Status and diff (same commands)
git status
git diff
```

```
# 7. Stage and commit
git add .
git commit -m "Add email validation to user registration"

# 8. Push
git push -u origin feature/add-email-validation
```

Pull Request targets `develop`, not `main`

```
base: develop  ←  compare: feature/add-email-validation
     ^^^^^^^^
     NOT main!
```

After merge

```
git checkout develop                          # NOT main
git pull origin develop
git branch -d feature/add-email-validation
```

How code flows to production in this model

```
your branch → PR → develop → (tested) → PR → main → (deployed)

You merge into develop.
Team lead merges develop into main when ready to release.
```

Side by Side Comparison

```
                    BRANCH OFF MAIN          BRANCH OFF DEVELOP
                    ________________         __________________

Start from:         git checkout main        git checkout develop
Pull latest:        git pull origin main     git pull origin develop
Create branch:      git checkout -b feat/x   git checkout -b feat/x
Work & commit:      (same)                   (same)
Push:               (same)                   (same)
PR target:          base: main               base: develop
After merge:        checkout main, pull      checkout develop, pull
```

Everything is the same except **which branch you start from and merge into**.

Scenario C: You Made Changes But Need to Switch Branches

You're working on your feature but need to quickly check something on main.

```
# Option 1: Stash your changes (temporary save)
git stash                    # saves your uncommitted work
git checkout main            # switch to main
# do whatever you need...
git checkout feature/x       # back to your branch
git stash pop                # restore your work

# Option 2: Commit what you have (even if incomplete)
git add .
git commit -m "WIP: email validation (in progress)"
git checkout main
# do whatever...
git checkout feature/x       # back to your branch, commit is there
```

Scenario D: Your Branch Is Behind (Someone Else Merged to Main)

You branched from main 2 days ago. Since then, a teammate merged their work.

```
main:        A ── B ── C ── D ── E ── F     ← teammate added E, F
                       |
your branch:           └── G ── H           ← your work (missing E, F)
```

You need to get their changes:

```
# On your feature branch
git checkout feature/add-email-validation

# Pull latest main and merge it into your branch
git fetch origin
git merge origin/main
```

If there are no conflicts:

```
Merge made by the 'ort' strategy.
 app/models/user.py | 5 +++++
 1 file changed, 5 insertions(+)
```

Now your branch has everything:

```
main:        A ── B ── C ── D ── E ── F
                       |             |
```

```
your branch:            └── G ── H ── M     ← M = merge commit (has E, F +
your G, H)
```

If there ARE conflicts:

```
CONFLICT (content): Merge conflict in app/routes/users.py
Automatic merge failed; fix conflicts and then commit the result.
```

Open the file:

```
<<<<<<< HEAD
    if not is_valid_email(user.email):
        raise HTTPException(422, "Invalid email")
=======
    if user.email in blocked_domains:
        raise HTTPException(422, "Blocked email domain")
>>>>>>> origin/main
```

You decide the right code (maybe keep both):

```
    if not is_valid_email(user.email):
        raise HTTPException(422, "Invalid email")
    if user.email in blocked_domains:
        raise HTTPException(422, "Blocked email domain")
```

Then:

```
git add app/routes/users.py
git commit -m "Merge main into feature branch, resolve conflict in
users.py"
```

Scenario E: "Oh No I Committed to Main by Accident"

```
# You're on main and accidentally committed
# Don't panic.

# Undo the commit (keep your changes)
git reset --soft HEAD~1

# Now create the branch you should have been on
git checkout -b feature/what-i-meant-to-do
```

```
# Your changes are still staged, commit them here
git commit -m "Add email validation"

# Main is back to normal, your work is on the right branch
```

Quick Decision Tree

```
"Where do I branch from?"

  Ask your team. Then:

  Team says "main"     → git checkout main && git pull origin main
  Team says "develop"  → git checkout develop && git pull origin develop
  Team says "staging"  → git checkout staging && git pull origin staging

  Then always:         → git checkout -b feature/your-task-name

"Where do I open my PR to?"

  Same branch you started from.
  Branched from main?    → PR into main
  Branched from develop? → PR into develop
```

When in doubt, `git status` is your best friend. Run it before and after everything.